Ansible-playbook Syntax

Task YML File: (Create a Directory)

```
- name: create a directory in /home
  file:
    path: /home/TestDir2
    state: directory
    owner: root
    group: root
    mode: 0755
  tags: [create_dir]
```




Exercise 4: create a play-book to create a play-book!

1. Create a playbook to setup the following structure on your ansible server (localhost)

```
.
├── inventory
│   └── anisahosts
├── myproject.yml
└── roles
    └── myproject
        ├── defaults
        │   └── main.yml
        ├── files
        ├── handlers
        │   └── main.yml
        ├── meta
        │   └── main.yml
        ├── tasks
        │   └── main.yml
        ├── templates
        └── vars
            └── main.yml
```




Run ansible-playbook

ansible-playbook -i inventory/myhosts myproject.yml

switches :	
-v/-vv/-vvv:	verbose for view Details
--tags:	"tag_name", Run Only tag_name used into task/main.yml
--tags:	"tag_name1,tag_name2,...", Select tags for Run
--skip-tags:	"tag_name,...", Select tags for Not Run
--step:	confirm each task before running (yes/no/continue)
--list-tasks:	List Of Tasks in YML File Without Run
--extra-vars:	VarName="Value", Create Variable

Exercise 5: ansible-playbook

1. Setup a play-book to do the following tasks on the two target hosts:
   1. Create dir1 and dir2 directories in /home
   2. Create file1 in dir1 and file2 in dir2
2. Check the syntax of your playbook
3. Run the playbook without any effect on the target host
4. List the tasks of the playbook
5. Run the playbook step by step and show the diff
6. Run it again and skip the steps which create directories

```
ansible-playbook -i inventory/hosts myproject.yml --syntax-check
ansible-playbook -i inventory/hosts myproject.yml --check
ansible-playbook -i inventory/hosts myproject.yml --list-tasks
ansible-playbook -i inventory/hosts myproject.yml --step --diff
```



```
ansible-playbook -i inventory/hosts myproject.yml -- skip-tags "create_dir2"
```

Variables

Use Variables to:

- Reduce redundancy
- Store a list of values and loop it
- Store user values using a prompt
- Specify the package name when running different OS
- Execute tasks and playbooks on multiple different systems with a single command.



Variable names

Invalid Variable names:

- Multiple words — mysql version
- A dot — mysql.port
- A number — 5
- A hyphen — user-input
- Reserved Keys — (python/playbook keywords)

✓ In many programming languages, variables that begin with an underscore are private. This is not true in Ansible. Variables that begin with an underscore are treated exactly the same as any other variable.



Variables in task files

کیی از جاهایی که می توانیم var تعریف کنیم

Example:

اعلان متغیر — اسم متغیر — مقدار

```
- set_fact: package_name=httpd
  when: ansible_os_family=="Redhat"

- set_fact: package_name=apache2
  when: ansible_os_family=="Suse“

- debug: msg={{ package_name }}

- name: install {{package_name }} package
  zypper:
   name: '{{ package_name }}'
   state: present
  tags: [Install_Apache]
```




Variable names

Invalid Variable names:

- Multiple words mysql version
- A dot mysql.port
- A number 5
- A hyphen user-input
- Reserved Keys (python/playbook keywords)

✓In many programming languages, variables that begin with an underscore are private. This is not true in Ansible. Variables that begin with an underscore are treated exactly the same as any other variable.



Variables

Use Variables to:

- Reduce redundancy
- Store a list of values and loop it
- Store user values using a prompt
- Specify the package name when running different OS
- Execute tasks and playbooks on multiple different systems with a single command.



Variables in playbook

Example:

```
---
- hosts: test-hosts
  vars:
   - package_name: “httpd”
  roles:
   - myproject
```




Variables as key:value dictionaries

✓A YAML dictionary maps keys to values.

Vars/main.yml:

```
lotus:
  env:
    version: '1.3.3.0'
    name:  'TEST'
  deployment:
    url: 'http://centdns1.lotus.ir/deployment'
    username: 'coreuser'
    password: ‘123456789'
  override:
    jms:
      url: 't3://localhost:7001'
```




Variables in vars files

tasks/main.yml:

```
- name: Download lotus core jar file
        get_url: url = {{ lotus.deployment.url }}/main-with-all-
dependencies.jar url_username = {{ lotus.deployment.username }}
url_password = {{ lotus.deployment.password }} dest = /var/lotus/libs/
main-with-all-dependencies-{{ lotus.env.name }}.jar mode = 0444
 notify: restart lotus core
 tags: [ 'lotus_core' ]
```


Copyright © 2020 Anisa Co.

List variables

✓ A list variable combines a variable name with multiple values.

Vars/main.yml:

```
User_list:
    - User1
    - User2
    - User3
```

Or

```
User: [user1,user2,user3]
```

✓ Referencing list variables

```
- debug:
    msg: "{{ user_list[0] }}"
```




Variables in global files

myproject.yml:

```
---
- hosts: test-hosts
  vars_files:
    - var1.yml
    - var2.yml
    - var3.yml
  tasks:
    - name: Start Apache service
      service: name={{ package_name }} state=started
```



Check priority



Command-Line Variables (variables at runtime)

✓ key=value format

```
ansible-playbook -i inventory/myhosts myproject.yml --extra-vars "nodes=test-hosts
package_name=apache2"
```

✓ JSON string format

```
ansible-playbook -i inventory/myhosts myproject.yml --extra-vars '{"nodes":"test-hosts","
package_name":"apache2"}'
```

myproject.yml:

```
---
- hosts: {{ nodes }}
  vars_files:
    - var1.yml
    - var2.yml
    - var3.yml
  tasks:
    - name: Start Apache service
      service: name={{ package_name }} state=started
```




Built-in Variables

Parameter	Description
hostvars	A dict whose keys are Ansible host names and values are dicts that map variable names to values
inventory_hostname	Name of the current host as known by Ansible
group_names	A list of all groups that the current host is a member of
groups	A dict whose keys are Ansible group names and values are a list of hostnames that are members of the group
play_hosts	A list of inventory hostnames that are active in the current play
ansible_version	A dict with Ansible version info



Exercise 6: ansible variables

1. Define a variable named myvar in vars section in my project.yml
2. Set myvar in tasks/main.yml → اولویت اول
3. Set myvar in vars/main.yml → دوم
4. Set myvar in vars/vars1.yml and vars/vars2.yml and use them as var_files → سوم
5. Set myvar in command-line
6. Check the priority by running the playbook

* بیشترین اولویت Runtime

Copyright © 2020 Anisa Co.



Command-Line Variables (variables at runtime)

✓ Values passed in using the key=value syntax are interpreted as strings. Use the JSON format if you need to pass non-string values such as Booleans, integers, floats, lists, and so on.

ansible-playbook -i inventory/myhosts myproject.yml -e '{"user_list":["user1","user2","user3"]}'

Tasks/main.yml:

```
---
- debug:
  msg:
   - "{{ user_list[0] }}"
```

ansible-playbook -i inventory/myhosts myproject.yml -e '{"nodes":["192.168.56.2","192.168.56.3"]}'

My project.yml

```
---
- hosts: "{{ nodes }}"
  roles:
   - myproject
```




Exercise 6: ansible variables

1. Define a variable named myvar in vars section in my project.yml
2. Set myvar in tasks/main.yml
3. Set myvar in vars/main.yml
4. Set myvar in vars/vars1.yml and vars/vars2.yml and use them as var_files
5. Set myvar in command-line
6. Check the priority by running the playbook



Defining variables in inventory - Host variables

✓ You can define different variables for each individual host, or set shared variables for a group of hosts in your inventory.

✓ Assign a variable to a single host

اعمال یک متغیر روی یک host

host برای inventory توی فایل

می توانیم یک متغیر ست کنیم

Inventory/anisahosts

```
[myservers]
192.168.56.2 http_port=80 myvar=2
192.168.56.3 http_port=8080 myvar=3

Or

[myservers]
192.168.56.2 ansible_connection=ssh ansible_user=anisa ansible_become_password=qazwsx
192.168.56.3 ansible_connection=ssh ansible_user=devops ansible_become_password=qaz
```




Exercise 7: non-standard SSH ports per host

1. Configure host1 to listen on port 723 as the non-standard ssh port
2. Allow port 723 on firewall
3. On your Ansible server define host1 in inventory host file so you can connect to the host on port 723
4. Run a sample playbook and test it!

Inventory/anisahosts

```
192.168.56.2:723

or

192.168.56.3 ansible_port=723
```




Variable merging

✓ By default variables are merged/flattened to the specific host before a play is run.

✓ By default, Ansible overwrites variables including the ones defined for a group and/ or host in the following order:

- all group
- parent group
- child group
- host

✓ By default Ansible merges groups at the same parent/child level in ASCII order, and the last group loaded overwrites the previous groups.

✓ You can change this behavior by setting the group variable *ansible_group_priority* to change the merge order for groups of the same level.



Variable merging

```
[Group_A]
192.168.56.2
192.168.56.3 http_port=8081

[Group_B]
192.168.56.2
192.168.56.3
192.168.56.4

[Group_A:vars]
http_port=8080
ansible_group_priority=7

[Group_B:vars]
http_port=80
ansible_group_priority=3

[anisaservers:children]
Group_A
Group_B
```

```
ansible-playbook -i inventory/anisahosts myproject.yml --tags show_var

TASK [myproject : ----- show http_port -----] ************************
ok: [192.168.56.2] => {
  "msg": 8080
}
ok: [192.168.56.3] => {
  "msg": 8081
}
```

host var

Group var

اگر یک host ما عضو چند Group باشد

با ansible_group_priority=7 اولویت را مشخص می‌کنیم

بعد از Group ها Groups of Groups هست

و در آخر All groups



Using multiple inventory sources

امکان داشتن چند تا host inventory

✓ You can target **multiple inventory** sources at the same time by giving multiple inventory parameters from the command line or by configuring ANSIBLE_INVENTORY. This can be useful when you want to target normally separate environments, like staging and production, at the same time for a specific action.

```
# ansible-playbook -i staging -i production myproject.yml
```

چند تا inventory ست شده

✓ if there are variable conflicts in the inventories, the merging order is controls the issue by the order of the inventory source parameters.

✓ For example in the above inventory order, the production variables will use but it the result would be reversed in the following run:

```
# ansible-playbook -i production -i staging myproject.yml
```




Command Module

ماژول Command هر دستوری که قابل اجرا باشه به جزء متغیرهای خاص و متغیرهای shell

- Takes the command name along with the arguments.
- shell variables or operations such as <, >, |, and & will not work as it will not be processed by the shell.

```
- name: command module
  command: cat TestFile
  args:
   chdir: /home/TestDir
   creates: /home/TestDir3
   removes: /home/TestDir2
  register: catty
  tags: [command_mod]

- debug: msg={{ catty.stdout }}
```

When it already exists, the command will not be run

When it does not exist, the command will not be run



Exercise 8: ansible command module

1. Use ansible command module to run the following commands
    1. Cat /home/dir1/file1
    2. df -h
    3. ps aux | grep ansible
2. Print the output of the above commands



expect Module

- The ***expect*** module executes a command and responds to prompts.
- shell variables or operations such as <, >, |, and & will not work as it will not be processed by the shell.
- Requirements:
    - Python >=2.6
    - Pexpect >= 3.3
- On the hosts:

Yum install epel-release
Yum install python-pip
Pip install pexpect
Or
apt-get install python3-pexpect

```
- name: expect module
  expect:
    command: passwd LPI
    responses:
      password: "Anisa"
  no_log: true
```

The command to run

Mapping of expected string to respond with

